KERALA DIPLOMA CURRICULUM REVISION 2026

Course 2132 | Semester II | Theory | CIE/ESE 40/60

Programming in C

Structured programming, control flow, arrays, functions, pointers and file handling.

OFFICIAL SOURCE DECLARATION

How this lesson was prepared

The supplied official syllabus PDF for Course 2132 is the authoritative source for course information, objectives, course outcomes, CO–PO mapping, modules, topics, activities, assessment structure and references. Explanations, examples and diagrams are educational elaboration and are not presented as additional official syllabus text.

Source note: The CO list mentions dynamic memory allocation, while the detailed topic table explicitly lists pointers and file operations. This lesson explains the memory concept carefully and identifies the distinction rather than silently changing the syllabus.

COURSE DASHBOARD

At a glance

Official course information

Item	Official information
Programme	Artificial Intelligence; Artificial Intelligence & Machine Learning; Cyber Forensics and Information Security; Computer Science and Technology; Computer Science & Engineering; Computer Engineering; Information Technology; Robotic Process Automation.
Teaching scheme	4:0:0:0 (L:T:P:R)
Contact hours	60 contact hours
Credits	4

Item	Official information
Self-learning	6/week; 90 hours listed in the PDF
Prerequisite	Basic problem solving and programming concepts; Course 1131 Problem solving and Python programming, Semester I.
Assessment	CIE 40 + ESE 60

Study sequence

Read the module introduction, learn each topic card, reproduce the diagram or procedure, then attempt the module questions.

Answer strategy

For a 1-mark answer define the term; for 3 marks add points; for 7 marks use a structured explanation, table, diagram or procedure.

Technical discipline

Retain official spellings, units, symbols, names, dates and distinctions when writing examination answers.

HOW TO USE THIS LESSON

A four-pass study method

Pass 1

Read the overview and identify the vocabulary of the module.

Pass 2

Study every open topic card and make a one-page notebook summary.

Pass 3

Practise the worked examples, procedures, sketches and comparisons.

Pass 4

Use the Q&A, model paper and quick revision section under timed conditions.

OUTCOMES AND ALIGNMENT

Objectives, COs and CO–PO mapping

Official course objectives

- Apply logical thinking and problem-solving techniques to real-world problems.
- Write efficient and structured C programs using appropriate language constructs.
- Build programming foundations for advanced programming languages.
- Familiarize with basic data structures and memory concepts for further study.

Course Outcomes

Course outcomes

CO	Outcome	Bloom level
CO1	Use sequential, conditional and looping structures in C.	Apply
CO2	Develop programs using arrays, matrices and strings.	Apply
CO3	Develop modular problem-solving skills using functions and structures.	Apply
CO4	Study pointers, files and dynamic memory allocation and develop programs using pointers and files.	Apply

CO–PO matrix

CO	mapped POs
CO1	PO1=2, PO2=3, PO3=3, PO4=3, PO7=2, PO11=2
CO2	PO1=3, PO2=3, PO3=3, PO4=3, PO7=2, PO11=2
CO3	PO1=3, PO2=3, PO3=3, PO4=3, PO7=2, PO11=2
CO4	PO1=3, PO2=3, PO3=3, PO4=3, PO7=2, PO11=2

COVERAGE CONTROL

Complete official syllabus coverage

Every official module and topic appears in the teaching cards below. Use this table as a checklist before the examination.

Module coverage

Module	Official syllabus items represented	Hours
I	Character set; tokens; constants; variables; identifiers; keywords; data types; initialization; operators; precedence; expressions; printf/scanf; program structure; decisions; loops; break/continue.	17 L
II	1-D/2-D arrays; searching; sorting; matrix representation; strings and library functions.	15 L
III	Functions; prototypes; calls; parameters; arrays as parameters; recursion; structures; unions.	15 L
IV	Pointers; pointer parameters; pointer applications; files; text/binary types; modes; opening/closing; reading/writing.	13 L

LEARNING ROADMAP

How the modules connect

1. Syntax

Tokens, variables and expressions create valid statements.

2. Control

Decisions and loops turn statements into algorithms.

3. Data and modules

Arrays, strings, functions and records organise information.

4. Memory and persistence

Pointers and files connect programs with memory and stored data.

MODULE 1

Basics of C Programming & Control Statements

Build a reliable mental model of C syntax, data, input/output and control flow.

17 L
official hours

CO1 / Apply
CO/Bloom

Learning goals

- Recognise tokens and declare variables correctly.
- Translate a flowchart or algorithm into C statements.
- Select if, switch or an appropriate loop for a problem.

Key facts

- C is case-sensitive.
- Every loop needs initialization, condition and progress.
- Formatted I/O depends on matching types and specifiers.

1. Character set, tokens and identifiers

A C program is built from a defined character set and lexical units called tokens.

Concept and explanation

The character set includes letters, digits, whitespace and special symbols. Tokens are the smallest meaningful units: keywords, identifiers, constants, string literals, operators and punctuators. An identifier names an object such as a variable or function; it must not start with a digit, contain spaces or be a keyword. C is case-sensitive, so total and Total are different identifiers.

Study points

- Use meaningful names such as totalMarks and itemCount.
- Keep declarations before use and initialise variables when a safe initial value is known.
- Do not use reserved words such as int, if, while or return as identifiers.

Engineering or professional relevance

Good naming makes debugging, maintenance and team review easier. In a larger program, a name communicates the role of a value before a reader studies the calculation.

> **Common mistakes:** Confusing a character constant with a string literal, using a keyword as a variable name, or assuming C ignores letter case.

> **Malayalam support:** C program □□□□□□□□□□ □□□ token-□□□ □□□□□□□□ □□□□□□□□□□□. variable name □□□□□□□□□□ □□□□□□□□□ code □□□□□□□□□□□ error □□□□□□□□□□□ □□□□□□□□□□.

2. Constants, variables, data types and initialization

A variable is a named memory location whose stored value may change during program execution.

Concept and explanation

The declared type determines the kind of value, memory interpretation and operations permitted. Common introductory types are int, float, double and char. A declaration introduces the name and type; initialization assigns the first value. Constants may be written as integer, floating, character or string literals. Choose a type that represents the expected range and precision.

Study points

- Declaration: int count;
- Initialization: int count = 0;
- Use %d for int, %f for floating output and %c for a character with the appropriate I/O statement.
- Avoid reading a value into an uninitialised or incompatible object.

Engineering or professional relevance

Data typing prevents many logical errors in sensor readings, counters, marks, prices and control signals. It also helps the compiler detect incompatible operations.

> **Common mistakes:** Assuming every numeric literal is a float, using a wrong format specifier, or dividing two integers when a fractional result is required.

> **Malayalam support:** Variable □□□□□□ value □□□□□□□□□□□□□ named memory location □□□. data type □□□□□□□□□□□□□□□□□□ value-□□□□ □□□□□□□□□ range-□□ □□□□□□□□□□□□□□□□.

3. Operators, precedence and expressions

Operators combine operands to form expressions that calculate, compare or modify values.

Concept and explanation

Arithmetic operators perform calculation; relational operators compare; logical operators combine conditions; the conditional operator selects one of two expressions; unary operators act on one operand; increment/decrement change a value by one; assignment operators store a result. Precedence determines grouping, but parentheses are the safest way to communicate intent.

Study points

- Separate calculation from decision-making when an expression becomes difficult to read.
- Use && for AND, || for OR and ! for NOT.
- Check integer division, remainder and the effect of pre-increment versus post-increment.

Engineering or professional relevance

Expressions appear in billing, control thresholds, array indices, engineering formulas and state transitions. Clear grouping reduces silent errors.

Operator groups

Group	Examples	Purpose
Arithmetic	+ - * / %	Numeric calculation
Relational	< <= > >= == !=	Comparison
Logical	&& || !	Combine conditions
Assignment	= += -= *= /=	Store/update value

Common mistakes: Writing = when == is needed, relying on precedence without parentheses, and changing a variable twice in one expression without tracing the sequence.

Malayalam support: Operator precedence □□□□□□□□□□□□□□ brackets □□□□□□□□□□□□. □□□ expression-□□□□ result □□□□□□ □□□□, evaluation order-□□ □□□□□□□□□□□□□□.

4. Formatted input and output

The standard functions printf() and scanf() connect a program to formatted text input and output.

Concept and explanation

printf() displays text and values according to format specifiers. scanf() reads input into variables and normally needs the address operator & for ordinary scalar variables. The format must match the declared type. A prompt, a clear input sequence and an output label make a program usable.

Study points

- Use printf("Enter marks: "); before scanf().
- Use &variable in scanf() for int, float and char scalar variables.
- Validate input where a wrong value could change the engineering decision.

Engineering or professional relevance

Console I/O is the first model of human-machine interaction. The same separation between input, processing and output appears in data acquisition and embedded systems.

Given: length and breadth are entered by the user.
Required: display rectangle area.
Calculation: area = length × breadth; in C, `area = length * breadth;`.
Answer: print the calculated area with its unit.

Common mistakes: Omitting &, mismatching %d/%f/%lf, and assuming scanf() automatically rejects every invalid input.

Malayalam support: Input □□□□□□□□□□□□□□□ □□□□□□□ prompt □□□□□. format specifier data type-□□□□ match □□□□□□□□; □□□□□□□□□□□ unexpected result □□□□.

5. Structure of a C program and sequential logic

A structured C program usually contains preprocessor directives, declarations, the main() function, statements and a return statement.

Concept and explanation

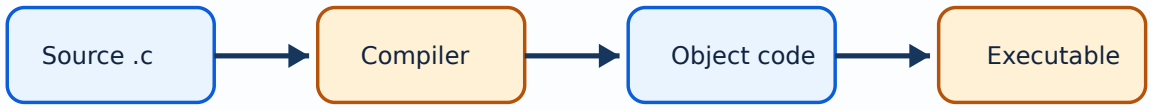
Execution begins in main() and proceeds sequentially unless a decision, loop or function call changes the flow. Comments document intent but do not execute. A small program should be decomposed into input, processing and output so that each stage can be checked independently.

Study points

- Include required headers before using library functions.
- Use braces and indentation consistently.
- Trace each statement with a small test input before adding complexity.

Engineering or professional relevance

Sequential programs model unit conversion, simple geometry, total-and-average calculations and calibration equations. They are the base from which control structures are built.



A schematic flow from source code through preprocessing, compilation, linking and execution.

Common mistakes: Writing statements outside a function, forgetting semicolons, and testing only one input value.

Malayalam support: C program-□□□□ flow □□□□□□□□□□ input → processing → output □□□□ □□□□□□□□□□□□. □ flow □□□□□ steps □□□ trace □□□□□□□□□□ □□□□□□□□ □□□□□□□□□□.

6. Decision control: if, if-else, nested if and switch

Decision statements select a path based on a condition.

Concept and explanation

if executes a block when its condition is true; if-else provides two alternatives; nested if handles layered decisions. switch is useful when one expression is compared against several constant case labels. Each case normally ends with break unless deliberate fall-through is required. A default case gives a safe response for unexpected input.

Study points

- Write the condition in plain language before coding it.
- Use braces even for a one-statement block.
- Order overlapping ranges carefully, for example grade bands from highest to lowest.

Engineering or professional relevance

Decision logic is used for alarm thresholds, grading, menu systems, machine states and safety interlocks.

Common mistakes: Using assignment inside a condition, omitting break in switch, and reversing > or < at a boundary.

Malayalam support: Condition □□□□□□□ □□□□□ □□□□□ □□□□□□□□□□ □□□□□. □□□□□□□□ □□□□□ if □□□□□□□□□□ switch □□□ code □□□□□□□.

7. Iterative control: while, do-while, for and nested loops

Loops repeat a block while a control condition permits it.

Concept and explanation

while checks before the first iteration; do-while executes once before checking; for groups initialization, condition and update in one header. A loop needs a correct initial value, a condition that can become false and an update that makes progress. Nested loops are useful for tables and matrices but multiply the number of executions.

Study points

- Trace the loop variable in a table for the first few iterations.
- Use for for counted repetition and while for condition-controlled repetition.
- Check off-by-one errors in ranges such as 1 to 100.

Engineering or professional relevance

Loops automate repeated measurement processing, table generation, searching, simulation steps and pattern printing.

Loop choice

Loop	First condition check	Typical use
while	Before body	Unknown number of repetitions
do-while	After body	Menu that must appear once
for	Before each iteration	Counted repetition

Common mistakes: Infinite loops, updating the wrong variable, and using <= when < is required for an array bound.

Malayalam support: Loop-□ initialization, condition, update □□□□□ □□□□□□ □□□□□□□□□ □□□□□□□□□□□□. update □□□□□□□□□□ loop □□□□□□□□□□□□ □□□□□.

8. break, continue and control-transfer discipline

break exits the nearest loop or switch; continue skips the rest of the current loop iteration.

Concept and explanation

These statements are useful when a search can stop early or invalid input should skip processing. They should clarify the algorithm rather than hide a complicated control flow. A flowchart often reveals whether a normal condition would be clearer.

Study points

- Use break when the answer is known and further iterations are unnecessary.
- Use continue only when the skipped path is obvious to the reader.
- Never use control transfer to bypass essential validation or resource closing.

Engineering or professional relevance

Early exit improves search efficiency; careful continue logic is useful in filtering sensor records or rejecting invalid data.

> **Common mistakes:** Breaking from the wrong nested loop, skipping the update before continue, and leaving an opened file unclosed.

> **Malayalam support:** break □□□□□□ nearest loop-□ □□□□□□ □□□□□□□ □□□□; continue current iteration □□□□□□ skip □□□□□□□.

1A. Algorithm, flowchart and pseudocode

Before writing C syntax, express the solution as an algorithm, flowchart or pseudocode.

Concept and explanation

An algorithm gives finite ordered steps; a flowchart shows control using standard symbols; pseudocode describes the logic without committing to a programming language. For a largest-of-three problem, identify inputs, comparisons, selected result and output before choosing nested if or a clearer sequence.

Study points

- State input, processing and output separately.
- Use a decision diamond for a condition and a process box for an assignment.
- Test the algorithm with equal, negative and boundary values.

Engineering or professional relevance

This method reduces syntax errors and gives a common communication language between a subject specialist and a programmer.

> **Common mistakes:** Starting to code before defining the condition, using ambiguous arrows, and omitting an end condition.

> **Malayalam support:** Algorithm □□□□□ □□□□□□□□ code-□□□□ logic □□□□□□□□□□□□. Flowchart-□ decision, process, input/output symbols □□□□□□□ □□□□□□□□□□□□.

1B. Implicit and explicit conversion

Expressions may involve operands of different numeric types.

Concept and explanation

Conversion can occur implicitly according to C rules, or explicitly using a cast. Converting a wider or fractional value to a narrower type can lose precision or range. In average calculations, convert at least one operand before division when integer truncation would be wrong.

Study points

- Identify the type of every operand.
- Use a cast only when its effect is understood.
- Do not hide a possible loss of precision in a long expression.

Engineering or professional relevance

Unit conversion, sensor scaling and percentage calculations often require deliberate type choice.

Given: total = 5 and count = 2 as integers.
Required: fractional average.
Calculation: `(float) total / count = 5.0/2 = 2.5`.
Answer: average = 2.5, not 2.

Common mistakes: Assuming 5/2 produces 2.5 in integer arithmetic, and casting after a value has already been truncated.

Malayalam support: Type conversion □□□□□□□□□□□□. Integer division-□ fractional part □□□□□□□□□□□; □□□□□□□□□□□□ float/double □□□□□□□□□□□.

1C. Nested decision and menu design

A nested decision is a decision inside another decision; a menu selects one operation from several choices.

Concept and explanation

Write the outer condition for the major classification and inner conditions for subcases. A menu driven program should display options, read a choice, execute a case and provide a default response. Separating input validation from the operation makes the design easier to test.

Study points

- Draw the decision tree before coding.
- Make boundary conditions mutually clear.
- Use switch for fixed choices and if for ranges.

Engineering or professional relevance

Menus appear in calculators, machine panels, diagnostics and laboratory utilities.

> **Common mistakes:** Overlapping ranges, missing default handling and deeply nested code that should be a function.

> **Malayalam support:** Nested decision-□ outer condition-□□□ inner condition-□□□ □□□□□□□□□ role □□□□. Menu-□ invalid choice handle □□□□□□□.

1D. Loop invariants and trace tables

A trace table records variable values after each iteration and exposes the invariant a loop maintains.

Concept and explanation

For a running sum, the invariant is that sum contains the total of all values processed so far. For a counter, the invariant states what has been counted. Trace tables are especially useful for nested loops, sorting and matrix algorithms.

Study points

- List the iteration number, index, condition and important variables.
- Check the first, middle and final iterations.
- State what is true before and after the loop.

Engineering or professional relevance

Trace reasoning is a basic debugging skill in control software and numerical processing.

Common mistakes: Tracing only the final value, ignoring the condition at termination and confusing current index with count.

Malayalam support: Trace table □□□□□□□□□□ □□□ iteration-□□□ variable values □□□□□□□□□□□□□□. Loop invariant □□□□□□□ □□□□□ □□□□□□□□□□□□.

Module tip: Link definitions to one worked example and one application before attempting long-answer questions.

MODULE 2

Arrays and Strings

Represent collections, tables and text using indexed memory.

15 L	CO2 / Apply
official hours	CO/Bloom

Learning goals

- Traverse one-dimensional and two-dimensional arrays.
- Write and trace search, sort and matrix algorithms.
- Handle strings with their terminator and library operations.

Key facts

- The first index is 0.
- Matrix operations have dimension rules.
- A string is a char array ending in \0.

9. One-dimensional arrays

An array stores a fixed number of values of the same type under one name and consecutive indices.

Concept and explanation

For an array declared as int marks[5], valid indices are 0 through 4. Declaration, initialization, access and traversal must agree on the length. Arrays support sum, average, maximum, minimum, counting and searching operations. The programmer must prevent out-of-bounds access.

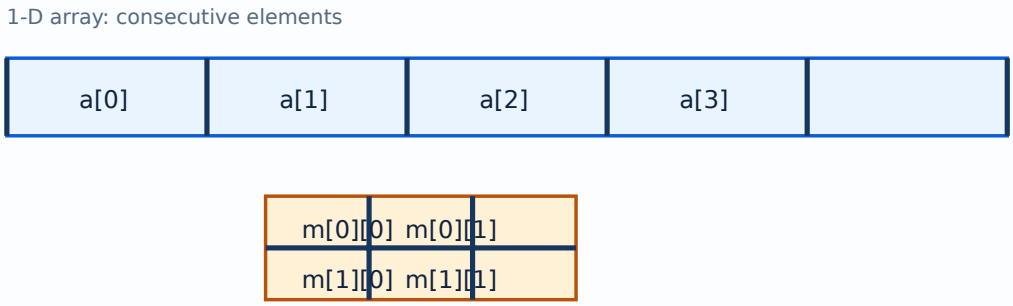
Study points

- Write the array length once using a constant where possible.
- Use a loop from index 0 to n-1.
- Draw an index-value table before debugging a traversal.

Engineering or professional relevance

Arrays represent repeated values such as readings, marks, component dimensions and daily production counts.

Given: values 4, 6, 8, 10.
Required: average.
Calculation: sum = 28; n = 4; average = 28/4 = 7.
Answer: 7.00.



One-dimensional arrays use one index; a matrix uses row and column indices.

Common mistakes: Starting at index 1, using n as an index, and reading more elements than were initialized.

Malayalam support: Array-□ □□□□ index 0 □□□. □□□□□□ n elements □□□□□□□□□□ □□□□□ index n-1 □□□□□□□□□□.

10. Searching and sorting arrays

Searching locates a value; sorting rearranges values into an order.

Concept and explanation

Linear search checks elements one by one and works on unsorted data. Bubble sort repeatedly compares adjacent elements and exchanges them when they are in the wrong order. At every pass, a largest remaining value moves toward the end. The lesson requires understanding the iteration, not memorising code only.

Study points

- Record the current index and comparison in a trace table.
- Stop linear search when the key is found.
- After each bubble-sort pass, verify the sorted suffix.

Engineering or professional relevance

Searching is used in inventories and lookup tables; sorting prepares records for reporting and can simplify later searches.

Algorithm comparison

Algorithm	Input assumption	Core operation
Linear search	No ordering needed	Compare key with each element
Bubble sort	Any order	Swap adjacent out-of-order values

Common mistakes: Comparing the wrong pair, forgetting to update the flag, and assuming bubble sort is efficient for every large data set.

Malayalam support: Search □□□□□□ value □□□□□□□□□□□□□□□ sort □□□□□□ order □□□□□□□□□□□□□□□. □□□ pass-□□□□ □□□□ array □□□□□□□□□□ □□□□□ □□□□□□.

11. Two-dimensional arrays and matrices

A two-dimensional array models rows and columns and is natural for matrix representation.

Concept and explanation

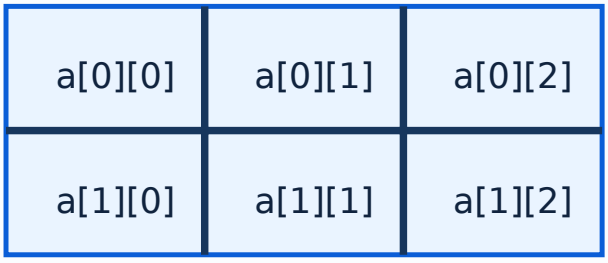
Declaration such as int a[2][3] creates two rows and three columns. Nested loops access a[i][j]. Matrix addition requires equal dimensions; multiplication requires the first matrix column count to equal the second matrix row count. A row-column diagram prevents index confusion.

Study points

- Use the outer loop for rows and inner loop for columns.
- State dimensions before writing the algorithm.
- Display matrices with aligned rows and columns.

Engineering or professional relevance

Matrices represent tabular measurements, transformation coefficients, adjacency data and image-like grids.



Row index is written first and column index second.

Common mistakes: Interchanging i and j, using incompatible dimensions, and forgetting to initialise the result matrix.

Malayalam support: Matrix-□ outer loop rows-□□□ inner loop columns-□□□ □□□□□□□□□□□□□□□ □□□□□□ □□□□ □□□.

12. Strings and library functions

In introductory C, a string is a character array terminated by the null character \0.

Concept and explanation

A string can be declared as char name[20] and initialized with a literal. Input and output require care because whitespace and buffer size affect behaviour. Common library functions include length, copy, concatenation and comparison functions from string.h. Always reserve space for the terminator.

Study points

- Count the maximum characters including the null terminator.
- Use comparison functions instead of comparing array names with ==.
- Explain what happens when the input is longer than the destination array.

Engineering or professional relevance

Strings are used for names, commands, file labels, product codes and messages in software systems.

> **Common mistakes:** Forgetting \0, copying into an undersized array, and treating a string as a single character.

> **Malayalam support:** String □□□□□□ characters-□□□□ array □□□; □□□□□□□□□□ null character □□□□□□□□□ □□□□□□□.

2A. Array declaration, initialization and traversal

Array work begins with a correct declaration and a disciplined traversal.

Concept and explanation

A declaration fixes the element type and capacity. Initialization may use a list of values or a loop. Traversal should use a valid index range and a separate logical length when fewer than the capacity slots are occupied.

Study points

- Keep capacity and current count conceptually separate.
- Use one traversal for one purpose where clarity matters.
- Check empty and full cases.

Engineering or professional relevance

Data logging systems frequently reserve an array and then process only the valid readings.

> **Common mistakes:** Using uninitialized slots, writing beyond capacity and treating capacity as the number of valid entries.

> **Malayalam support:** Array capacity-□□□ current count-□□ □□□□□□□□□□□□ □□□□□□□□□□□□. Valid index range 0 □□□□ n-1 □□□ □□□.

2B. Array statistics and frequency counting

Sum, average, maximum, minimum and frequency counts are common array-processing patterns.

Concept and explanation

A statistic algorithm needs an initialization strategy. Sum starts at zero; maximum/minimum often start from the first valid element rather than an arbitrary constant. Frequency counting increments a counter when a condition matches.

Study points

- Handle the zero-element case explicitly.
- Use the correct comparison for maximum/minimum.
- Explain whether repeated values are counted once or per occurrence.

Engineering or professional relevance

These patterns support marks analysis, production inspection and measurement summaries.

> **Common mistakes:** Initializing maximum to zero when all values may be negative, and dividing by zero for an empty collection.

> **Malayalam support:** Maximum/minimum initialize □□□□□□□□□□□ data range □□□□□□□□□□□. Empty array case □□□□□□□□□ handle □□□□□□□.

2C. Bubble-sort passes and complexity intuition

Bubble sort is learned by tracing adjacent comparisons and swaps pass by pass.

Concept and explanation

At the end of a complete pass, the largest unsorted element reaches the right side. The algorithm may stop early if a pass makes no swap. Although simple for teaching, repeated comparisons make it less suitable for large collections than more advanced methods.

Study points

- Write the array after every pass.
- Use a swap flag for the already-sorted case.
- Distinguish ascending and descending comparison.

Engineering or professional relevance

The pass invariant helps students understand why a loop inside a loop is needed.

Common mistakes: Stopping after one swap, comparing non-adjacent elements without a reason and forgetting that the sorted boundary grows.

Malayalam support: Bubble sort-□ □□□ pass-□□□□ □□□□□□ largest unsorted value right side-□□□□□□ □□□□□.

2D. Matrix addition and multiplication

Matrix algorithms require dimensions, row/column indexing and a correctly initialized result.

Concept and explanation

For addition, corresponding entries are combined. For multiplication, each result entry is the dot product of one row and one column. Three nested loops are commonly needed: result row, result column and shared dimension.

Study points

- Write the dimension rule before the code.
- Initialize each result element before accumulating.
- Trace one result entry by hand.

Engineering or professional relevance

Matrix operations occur in graphics, transformations, tabular engineering calculations and network representations.

> **Common mistakes:** Using addition dimensions for multiplication, forgetting to reset the result and swapping row/column indices.

> **Malayalam support:** Matrix multiplication-□ row × column dot product □□□. Shared dimension match □□□□□□□.

2E. String input, comparison and custom operations

String processing combines array bounds, character traversal and the null terminator.

Concept and explanation

Library functions simplify length, copy, concatenate and compare operations, while custom loops reveal how those operations work. Input with spaces requires a method that preserves the intended text and does not overflow the destination array.

Study points

- Reserve one position for \0.
- Stop a traversal at \0 or the stated bound.
- Compare contents, not array addresses.

Engineering or professional relevance

Command interpreters, labels and user records all depend on safe text processing.

Common mistakes: Buffer overflow, comparing with == and copying without a destination-size check.

Malayalam support: String library functions □□□□□□□□□□□□□ size, null terminator, input safety □□□□□□ □□□□□□□□□□□□.

Module tip: Link definitions to one worked example and one application before attempting long-answer questions.

MODULE 3

Functions and Structures

Decompose programs into reusable functions and meaningful records.

15 L	CO3 / Apply
official hours	CO/Bloom

Learning goals

- Write prototypes, calls and definitions.
- Explain parameter passing and recursion.
- Model records using structures and distinguish unions.

Key facts

- A prototype prevents type ambiguity.
- Recursion needs a base case.
- Structures store all members separately; unions share storage.

13. Functions: prototype, definition, call and parameters

A function is a named reusable block that performs a focused operation.

Concept and explanation

A prototype tells the compiler the function name, return type and parameter types. The definition contains the body; a call transfers control to it. Pass by value gives a copy, while pointer-based passing allows a function to modify an object through its address. Arrays passed to functions are handled through their address and a separate length is normally supplied.

Study points

- Give a function one clear responsibility.
- Match argument types with parameter types.
- Use a return value for a calculated result and pointer parameters for multiple outputs.

Engineering or professional relevance

Functions support modular testing, reuse and team development. In engineering software they separate acquisition, conversion, validation and reporting.

> **Common mistakes:** Calling before a visible prototype, returning the address of a local variable, and assuming pass by value changes the caller’s variable.

> **Malayalam support:** Function code-□□ □□□□□ □□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□ testing □□□□□□□□□□□□. parameter passing □□□□□□□□□□□□□□□ □□□□□□□.

14. Recursion

Recursion solves a problem by calling the same function on a smaller instance until a base case is reached.

Concept and explanation

A recursive definition needs a base case and a recursive case that makes progress toward it. Factorial and Fibonacci are familiar examples, but recursion can use more memory than an iterative solution because calls are stored on the call stack. Trace the call and return sequence for small inputs.

Study points

- Write the base case first.
- Confirm that each call reduces the problem.
- Compare recursive and iterative approaches for efficiency and clarity.

Engineering or professional relevance

Recursion is useful for tree-like structures, divide-and-conquer algorithms and nested problem representations.

> **Common mistakes:** Missing the base case, changing the argument in the wrong direction, and ignoring stack depth.

> **Malayalam support:** Recursion-□ base case □□□□□□□□□□□. □□□ call-□□ problem □□□□□□□□□□□□□□□□□□ □□□□□ □□□□□□□□□□□□.

15. Structures and unions

A structure groups related values that may have different types under one name. A union shares storage among members.

Concept and explanation

A structure is suitable for a record such as a student, component or product because all members can hold values simultaneously. A union stores one active member meaningfully at a time, so its memory use is different. Define the type, declare a variable and access members with the dot operator.

Study points

- Use structures for records with simultaneous fields.
- Use arrays of structures for multiple records.
- Explain the storage difference when comparing union and structure.

Engineering or professional relevance

Structures model engineering records such as a machine part with id, mass and dimensions; they make data handling clearer than unrelated variables.

Structure versus union

Feature	Structure	Union
Storage	Separate storage for members	Shared storage
Simultaneous values	All members can be valid	One member is meaningful at a time
Use	Record modelling	Memory-saving alternative when appropriate

Common mistakes: Confusing a type definition with a variable declaration, using the wrong member operator, and reading an inactive union member as if it were current.

Malayalam support: Structure-□ □□ members □□□□□□□□□ value □□□□□□□□□□□; union-□ □□□ memory □□□□□□□□□□□□□□□ active member □□□□□□□□□□□□.

3A. Return values and local scope

A function’s return value communicates one calculated result to its caller.

Concept and explanation

Local variables exist within their block or function and cannot be used directly outside their scope. Parameters provide input; a return statement provides output. A function should not rely on hidden global state when a parameter makes the dependency explicit.

Study points

- Name the return type according to the result.
- Do not use a local variable after its lifetime.
- Keep side effects visible and limited.

Engineering or professional relevance

Scope discipline makes larger programs testable and prevents accidental interference between modules.

> **Common mistakes:** Returning no value from a non-void function and expecting a local variable to remain accessible after return.

> **Malayalam support:** Local variable-□□□□ scope function/block-□ □□□□□□□□□□. Return value caller-□□ result □□□□□□□□.

3B. Call by value and pointer-based modification

C passes ordinary arguments by value, so the called function receives a copy.

Concept and explanation

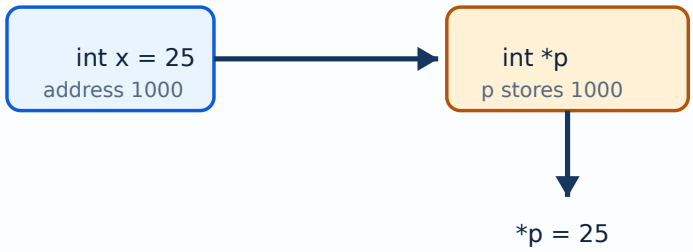
To modify a caller’s object, pass its address and use a pointer parameter. A swap function demonstrates the distinction: swapping local copies does not change the caller, while swapping through addresses does.

Study points

- Draw caller variables and parameter variables separately.
- Use & at the call and * in the function when appropriate.
- Check pointer validity before dereferencing.

Engineering or professional relevance

Pointer-based parameters allow a function to return multiple results or update an array, record or status flag.



A pointer stores an address; dereferencing accesses the value at that address.

Common mistakes: Claiming C has direct call-by-reference syntax, using the wrong pointer type and dereferencing NULL.

Malayalam support: C-□ ordinary parameter copy □□□ □□□□□□□□. Caller value □□□□□□□ address/pointer □□□□□□□□□□□.

3C. Arrays as function parameters

When an array is passed to a function, the function receives access to its elements through an address-like parameter.

Concept and explanation

The function does not automatically know the valid length, so the length should be passed separately. A const qualifier communicates that a function will read but not modify an array.

Study points

- Pass both array and logical length.
- Use const for read-only calculations.
- Test an empty or one-element array.

Engineering or professional relevance

This pattern separates data processing from the main program and supports reusable statistics or search functions.

> **Common mistakes:** Walking past the valid length, assuming the function receives a complete copied array and modifying data unintentionally.

> **Malayalam support:** Array function-□□□□□□ pass □□□□□□□□□□□ length □□□□□□□□□ □□□□□□□□□ safe □□□.

3D. Recursion trace and stack behaviour

Recursive calls create a stack of pending work that unwinds after the base case.

Concept and explanation

For factorial, each call records its argument and waits for the smaller result. The return sequence reverses the call sequence. This mental model is more important than memorising one program.

Study points

- Write the base case and recursive call on separate lines.
- Trace n=3 or n=4 on paper.
- Compare memory use with an iterative loop.

Engineering or professional relevance

The stack model prepares students for nested function calls and later data structures.

> **Common mistakes:** Assuming recursive calls return immediately, and allowing input to grow without considering stack depth.

> **Malayalam support:** Recursion call stack-□ pending calls □□□□□□□□□□□□. Base case □□□□□□□□□□□ □□□□□ reverse order-□ return □□□□□□□□.

3E. Array of structures

An array of structures stores multiple records with the same fields.

Concept and explanation

For example, an array of components may store code, mass and length for each component. Access requires two levels: record index and member name. A function can search or sort records using a chosen field.

Study points

- Define the record type once.
- Use a meaningful key field for search.
- Display headers and units when printing records.

Engineering or professional relevance

Record arrays are closer to real engineering data than parallel arrays with unrelated names.

> **Common mistakes:** Mixing the array index with the member access and sorting the wrong field.

> **Malayalam support:** Array of structures □□□□□□□□□□ □□ records-□□ □□□ fields □□□□□ □□□□□□□□□□□□.

3F. Modular testing and debugging

A modular program can be tested function by function before integration.

Concept and explanation

Use normal, boundary and invalid inputs. A test case should state the input, expected result and observed result. Compiler errors, run-time faults and logical errors require different debugging approaches.

Study points

- Test the smallest function first.
- Keep the first failing input.
- Use printf tracing carefully and remove diagnostic output when finished.

Engineering or professional relevance

Testing is essential in educational programs and production systems because a compiled program is not automatically a correct program.

Common mistakes: Testing only one happy-path input, changing several functions at once and failing to reproduce the error.

Malayalam support: Compile □□□ □□□□ alone correct □□□□□□□. Input, expected output, observed output compare □□□□□□ test □□□□□□□.

Module tip: Link definitions to one worked example and one application before attempting long-answer questions.

MODULE 4

Pointers and File Handling

Connect addresses, indirect access, file modes and persistent data.

13 L	CO4 / Apply
official hours	CO/Bloom

Learning goals

- Trace address and dereference operations.
- Choose a file mode and follow the open/use/close sequence.
- Relate the CO’s memory wording to the detailed pointer/file topics.

Key facts

- FILE * is a pointer to file-management state.
- A pointer is not the value it points to.
- Every resource opened should be closed.

16. Pointers, files and memory concepts

A pointer stores an address and can be dereferenced to access the object at that address. Files provide persistent storage beyond one program run.

Concept and explanation

Pointer declaration, address-of (&), dereference (*) and pointer parameters form the base of the topic. C file operations use a FILE pointer with modes such as r, w, a, r+, w+ and a+. The sequence is open, check, read/write, close. The syllabus CO mentions dynamic memory allocation; this lesson treats it as a memory concept while keeping the explicitly listed basic pointer and file operations central.

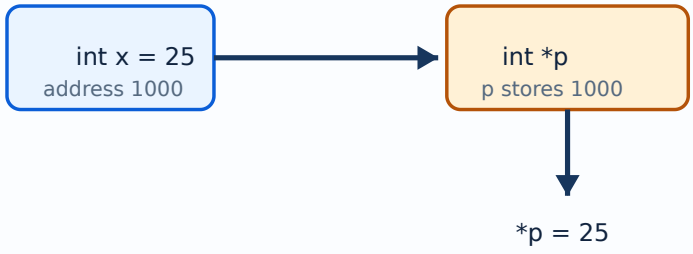
Study points

- Check a file pointer for NULL after fopen().
- Use the correct mode and close every opened file.
- Do not dereference an uninitialised or invalid pointer.

Engineering or professional relevance

Pointers support arrays, functions, dynamic data and low-level device interfaces; files support logs, reports and configuration data.

Given: a file must receive a report.
Required: safe sequence.
Calculation: fopen("report.txt","w") → check for NULL → fprintf() → fclose().
Answer: always check the open result and close the file after writing.



A pointer stores an address; dereferencing accesses the value at that address.

Common mistakes: Using * on an invalid address, mixing text and binary assumptions, forgetting fclose(), and overwriting a file with w when append was

intended.

Malayalam support: Pointer address സൂക്ഷിക്കുന്നു ; dereference [illegible] [illegible] address-[illegible] ഉള്ള value ലഭിക്കും . File workflow open → check → operate → close ആണ്.

4A. Address, pointer type and dereference

Pointer operations are safe only when type and lifetime agree with the target object.

Concept and explanation

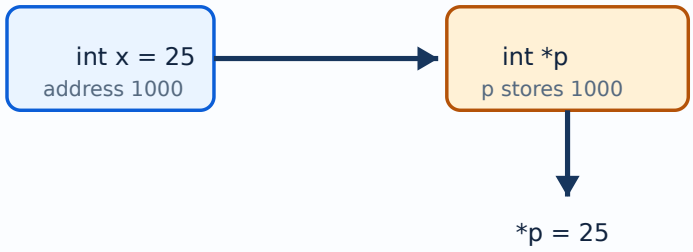
The address-of operator obtains an address; a pointer variable stores it; dereference accesses the pointed object. Pointer arithmetic is meaningful within an array object and advances according to the pointed type, not simply one byte in ordinary typed use.

Study points

- Initialize pointers to a valid address or NULL.
- Match pointer type with target type.
- Never dereference NULL or an expired object.

Engineering or professional relevance

Pointers underpin arrays, strings, dynamic data and low-level interfaces.



A pointer stores an address; dereferencing accesses the value at that address.

Common mistakes: Using an arbitrary integer as an address, returning an invalid local address and confusing pointer value with target value.

Malayalam support: Pointer type target object-□□□□ match □□□□□□□. NULL dereference □□□□□□□□□.

4B. Pointers and arrays

An array name is closely related to the address of its first element in expressions, but an array and a pointer are not identical objects.

Concept and explanation

Indexing can be understood through pointer movement: a[i] corresponds conceptually to *(a+i). This relationship explains array parameters and string traversal, while also requiring respect for bounds.

Study points

- Use indexing when it improves readability.
- Use pointer notation only when the address behaviour is understood.
- Keep the array length available.

Engineering or professional relevance

Understanding the relationship helps later study of strings, dynamic memory and data structures.

> **Common mistakes:** Incrementing a pointer outside the array, and assuming sizeof(pointer) equals the array storage size.

> **Malayalam support:** Array name-□□□ pointer-□□□ □□□□□□□□□□□□□□□□□ □□ same object □□□□. Bounds □□□□□□□□□.

4C. Pointer parameters and multiple outputs

A function can use pointer parameters to write more than one result for the caller.

Concept and explanation

For a division routine, one pointer may receive quotient and another remainder, while the function returns an error status. This separates success/failure from calculated outputs.

Study points

- Check every output pointer before writing.
- Document which parameters are input and which are output.
- Use a status return value consistently.

Engineering or professional relevance

Multiple-output functions are useful in parsing, measurement conversion and validation routines.

> **Common mistakes:** Writing to an uninitialised output pointer and ignoring the function’s status result.

> **Malayalam support:** Pointer parameter input □□□ output □□□ □□□□□□□□□□□□□□. Status return check □□□□□□□.

4D. Text and binary files

Text files store characters in a human-readable representation; binary files store bytes in a format intended for program use.

Concept and explanation

The choice affects portability, readability, size and processing. Text functions such as formatted input/output suit reports; binary operations require careful size and structure assumptions. Both require a correct mode and error checking.

Study points

- Choose text for human-readable reports.
- Document binary record layout.
- Close after every operation sequence.

Engineering or professional relevance

File choice matters in logs, configuration, reports and data interchange.

> **Common mistakes:** Opening a binary file as if it were formatted text, and assuming binary data is portable without documenting representation.

> **Malayalam support:** Text file □□□□□□□□ □□□□□□□□□; binary file byte representation □□□. Use-case □□□□□□□□□□ mode □□□□□□□□□□□□□□□.

4E. File modes and workflow

File mode determines whether a file is read, written, appended or updated.

Concept and explanation

r expects an existing file for reading; w creates or truncates for writing; a appends. Update modes combine reading and writing but require careful positioning and error handling. The safe workflow is open, check, operate, close.

Study points

- State whether existing content must be preserved.
- Check NULL immediately after opening.
- Use fclose even on an early-return path.

Engineering or professional relevance

Mode selection prevents accidental data loss in reports and logs.

> **Common mistakes:** Using w for a log that should append, omitting the mode, and closing only in the normal path.

> **Malayalam support:** Mode □□□□□□□□□□□□□□□□□□□ old content preserve □□□□□□□□□ □□□□□ □□□□□ □□□□□□□□□□□□□□.

4F. Reading and writing records

Formatted file I/O writes values as text according to a format; character or line operations process text incrementally.

Concept and explanation

A record-writing program should define the field order and delimiter so a later reader can interpret the file. Always report whether the operation succeeded and keep units in human-readable reports.

Study points

- Write a header for a report where useful.
- Use delimiters consistently.
- Test empty file and multiple-record cases.

Engineering or professional relevance

Persistent records support inventory, attendance, measurement and maintenance logs.

> **Common mistakes:** Reading fields in a different order than writing them and assuming every line is complete.

> **Malayalam support:** File record format □□□□□□□□□ □□□□□□□□□□□□□. Write order-□□□ read order-□□□ match □□□□□□□.

4G. Dynamic memory concept

The CO references dynamic memory allocation even though the detailed topic emphasis is pointers and files.

Concept and explanation

Dynamic memory is obtained during execution rather than fixed entirely at compile time. It must be checked, used within its allocated bounds and released when no longer needed. The principle connects pointer validity, lifetime and resource management.

Study points

- Distinguish stack, static and dynamically managed storage conceptually.
- Check allocation failure.
- Release memory once and do not use it afterward.

Engineering or professional relevance

Dynamic storage is important for data whose size is known only after input, but it introduces ownership and lifetime responsibilities.

> **Common mistakes:** Memory leak, double release and use-after-release.

> **Malayalam support:** Dynamic memory run time-□ □□□□□□□□□□□□. Allocate, check, use, release □□□□ lifecycle □□□□□□□□□.

4H. Integrated mini-project: file-backed marks report

Combine input, arrays, functions, structures and files in one small program design.

Concept and explanation

A marks report can read a count, store records, calculate totals or averages through functions, display a report and write the result to a text file. Design the data record and file format before coding.

Study points

- Separate input, calculation, display and file functions.
- Validate count and marks.
- Test at least one invalid input and one empty case.

Engineering or professional relevance

The project demonstrates how individual syllabus topics cooperate in an engineering-style utility.

> **Common mistakes:** Writing the whole program in main, mixing file I/O with calculation and failing to close the file.

> **Malayalam support:** Mini-project-□ functions, structures, arrays, validation, file writing □□□□□□ □□□□□□□□□□□ □□□□□□□□□□□.

> **Module tip:** Link definitions to one worked example and one application before attempting long-answer questions.

RAPID REFERENCE

Formula and concept bank

Average of values

```
average = sum / count
```

Use floating division when a fractional answer is required.

Array index

```
0 ≤ index < n
```

For n elements, the last valid index is n−1.

Matrix addition

```
C[i][j] = A[i][j] + B[i][j]
```

A and B must have equal dimensions.

Pointer access

```
value = *p
```

p must contain a valid address before dereferencing.

File workflow

```
open → check → read/write → close
```

Use the correct mode and check FILE * for NULL.

RAPID REVISION

Diagram library

C program execution flow

Use for Module I compilation sequence.

Array and matrix indexing

Use for Module II row/column reasoning.

Pointer relationship

Use for Module IV address tracing.

OFFICIAL SELF-LEARNING

Activities from the syllabus

Complete these activities as evidence of self-learning. Keep sketches, tables, screenshots, observations or short reports in a subject file.

1. Data/type/declaration table

Prepare a real-world table of values, types, variables and declarations.

2 hours

2. Execution and online compiler note

Record compilation and execution steps and compare two online C compilers.

2 hours

3. Algorithms and simple programs

Write algorithms and programs for 1–100, factorial, Fibonacci, even/odd and largest of three.

6 hours

4. Output prediction

Predict output or errors for short code snippets and explain each result.

3 hours

5. Array and matrix workbook

Create sum/average/max/min, bubble sort, linear search and matrix-addition traces.

12 hours

6. String practice

Write programs using string functions and a second version without library functions.

6 hours

7. Function and recursion file

Implement parameter-passing examples, recursion and arrays of structures.

12 hours

8. Pointer and file investigation

Draw addresses, prepare the file-mode table, diagnose missing fclose/wrong modes and write a file program.

24 hours

9. Online quizzes

Complete the syllabus-specified Google Form or equivalent quiz evidence.

3 hours

EXAMINATION PREPARATION

Assessment methodology

The official assessment is CIE 40 and ESE 60. The displayed structure includes attendance 5, four self-learning activity components and two series examinations converted as specified in the syllabus, followed by a 60-mark ESE.

Series examination

Duration: 2 hours. The paper uses the official 1/3/7-mark structure, with module-set choices as specified in the PDF and a total of 50 marks before conversion.

ESE

Duration: 2.5 hours. Questions are distributed across all four modules using Part A 1-mark, Part B 3-mark and Part C 7-mark components for a total of 60 marks.

Exam discipline: Do not label this lesson’s model paper as an official examination paper. It is a practice aid based on the official pattern.

ACTIVE RECALL

Module-wise questions and answers

Expand each question only after attempting it from memory. Match answer length to the mark value.

Module 1

What is a token in C?

A token is the smallest meaningful lexical unit, such as a keyword, identifier, constant, string literal, operator or punctuator.

Differentiate while and do-while.

while checks the condition before the first iteration; do-while executes the body once and checks after it.

What is operator precedence?

It is the rule that determines which operators are grouped first in an expression; parentheses should be used when clarity is important.

Write the safe stages of a sequential program.

Read input, validate it where necessary, perform the calculation, and display a labelled output.

Module 2

What is an array?

An array is a fixed-size collection of same-type elements accessed by index, beginning at index zero.

State one condition for matrix multiplication.

The number of columns in the first matrix must equal the number of rows in the second.

Why is the null character important?

It marks the end of a C string so library functions know where the text stops.

What is linear search?

It checks each element in sequence until the key is found or the array ends.

Module 3

What is a function prototype?

A declaration that gives the function name, return type and parameter types before the function is called.

What is recursion?

A function-solving technique in which a function calls itself on a smaller problem until a base case is reached.

Differentiate structure and union.

A structure gives members separate storage; a union shares storage and normally holds one meaningful member at a time.

Why pass an array length to a function?

The array parameter does not by itself communicate the number of valid elements, so the length prevents unsafe traversal.

Module 4

What is a pointer?

A pointer is a variable that stores the address of another object.

Name two file modes.

r opens for reading and w opens for writing, creating or truncating the file as defined by the C library.

What is dereferencing?

Using * with a valid pointer to access the value stored at its target address.

Why should fclose() be called?

It releases the file resource and ensures buffered output is flushed correctly.

PRACTICE ONLY

Practice Model Paper — Not an Official Examination Paper

Practice programs should be written by hand once, compiled once, and traced once. The model paper is not official.

Part A — 1 mark each

Answer all short definitions and identifications.

Part B — 3 marks each

Answer concise explanations, comparisons or procedures.

Part C — 7 marks each

Answer structured long questions with diagrams, tables, examples or applications.

Self-check

Reserve the last 10 minutes to check labels, units, terminology and question choices.

FINAL REVISION

Quick revision and exam checklist

- Module 1: revise the official headings, terms and one labelled diagram.
- Module 2: revise the official headings, terms and one labelled diagram.
- Module 3: revise the official headings, terms and one labelled diagram.
- Module 4: revise the official headings, terms and one labelled diagram.
- Practise at least one 1-mark, 3-mark and 7-mark answer.
- Read every official self-learning activity as a possible viva or assignment prompt.

Common mistakes

Do not confuse definitions with examples, omit labels from diagrams, copy a formula without symbol meanings, or write unsupported claims as official syllabus information.

Last-day checklist

Revise every module heading, formula/concept bank, diagram library, activity list, assessment pattern and at least one answer per mark category.

VOCABULARY

Glossary

Important terms

Term	Short meaning

Array average calculator

Comma-separated values 4,6,8,10

OFFICIAL RESOURCES

References

Textbook(s)

- Programming in ANSI C — E. Balagurusamy, McGraw-Hill; The C Programming Language — Kernighan & Ritchie, Pearson.

Reference books

- C Programming Absolute Beginner’s Guide — Greg Perry, Pearson
- Let Us C — Yashavant Kanetkar, BPB

Web resources listed in the syllabus

IN-PAGE SEARCH

Search this lesson

Prepared as a student-friendly REV2026 lesson from the supplied official syllabus PDF. Verify institutional updates against the current official curriculum.